

Sequential, Parallel and Distributed Solution of the Railroads Puzzle Using a Genetic Algorithm

Marija Dejkoska
University of Primorska
Koper, Slovenia
89231040@student.upr.si

ABSTRACT

This report represents a genetic algorithm for solving the Railroads puzzle. The program generates a railway board containing trains and destinations and searches for a tile configuration in which all trains can reach their assigned destinations. Three implementations are presented: sequential, parallel and distributed. The parallel version uses Java threads, whereas the distributed version uses MPJ Express. Their correctness and execution times are compared using boards of different sizes and workloads.

1 INTRODUCTION

The goal of Railroads is to solve a railway puzzle using a genetic algorithm. A board can contain several trains, each with its uniquely assigned destination. The algorithm modifies railway tiles until valid oaths between the trains and their destinations are found.

The project was implemented in three versions: sequential, parallel and distributed. The purpose of the parallel and distributed versions is to reduce the time needed to evaluate the population of candidate boards.

2 PROBLEM DESCRIPTION

2.1 Board Representation

Railroads is depicted as a two dimensional grid of railway tiles. Grid dimensions and number of train pairs (constituting of a train and its unique destination) are chosen by the user through the graphical interface.

Each cell of the grid has its own integer value. Tile types are represented by integer values from 0 to 10, negative values are used for trains, destinations and empty cells. An empty cell is represented by the value of -1, a train by -2, whereas destination is -3.

Each train has its own unique destination, both of their positions being assigned by a random fixed seed making it easier for testing. A train object stores the row and column of the starting position together with the row and column of its destination. Listing 1 shows the information stored for each train. The object contains the coordinates of its starting cell and its assigned destination.

```
1 public class Train {
2     public final int startingRow;
3     public final int startingColumn;
4     public final int destinationRow;
5     public final int destinationColumn;
6
7     public Train(int startingRow,
8                 int startingColumn, int destinationRow,
9                 int destinationColumn){
10         this.startingRow = startingRow;
11         this.startingColumn = startingColumn;
12         this.destinationRow = destinationRow;
13         this.destinationColumn =
14             destinationColumn;
15     }
16
17     public int getStartingRow(){
18         return startingRow; }
19     public int getStartingColumn(){
20         return startingColumn; }
21     public int getDestinationRow(){
22         return destinationRow; }
23     public int getDestinationColumn(){
24         return destinationColumn; }
25
26
27 }
```

Listing 1: Representation of a train and its destination.

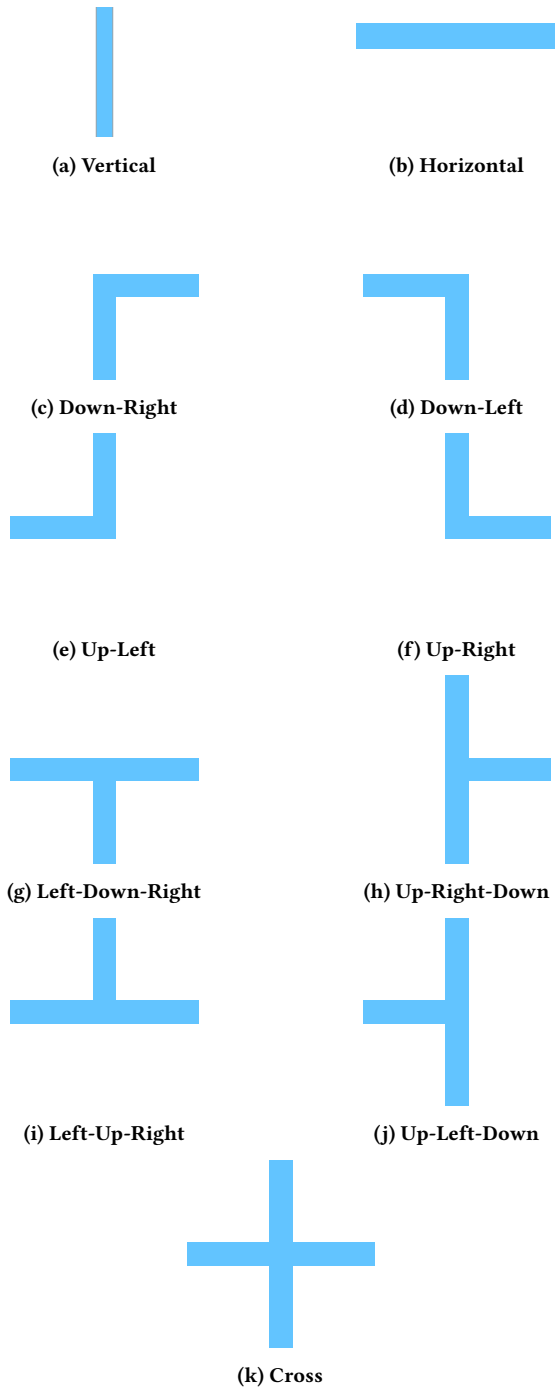
Two copies of the board are maintained. The initial board which is the original assigned puzzle before any modifications, and the current board which contains the changes made by the user or the genetic algorithm. The initial board is used as a reference during tile-change penalty calculations.

2.2 Tile Types

The board supports several railway tile types. These include both a horizontal and vertical straight tile, four two-directional corner tiles, four three-directional junction tiles, and a cross (four-directional).

Each tile defines the directions in which a train may enter or leave the cell. For example, a horizontal tile connects the left and right neighbors, the vertical tile connects the upper and lower neighbors. Corner tiles connect two perpendicular directions, and three-directional tiles connect three neighboring cells. The cross connects all four directions.

The cross guarantees that a valid solution can always be produced as it allows omnidirectional movement. The pitfall is the more neighbors the tile connects, the higher its assigned penalty.



This is to force the genetic algorithm to create simpler solutions, if they're producible.

2.3 Train Paths

A train may move from one cell to another adjacent cell only when both of the cells have compatible railway connections. For example, if the current tile has an open connection to the right, the tile in the next column must have an open connection to the left. The same rule applies to movement in all other directions.

Determining if a train has reached its destination is done through depth-first-search. The search starts from the train's initial position and follows valid railway connections only, marking visited cells so that the same cell is not visited multiple times.

A search is considered successful when it reaches the destination assigned to that train. This procedure is repeated independently for every train-destination pair on the board. A candidate board is considered solved only when all trains can reach their destinations.

```

1  public PathResult DFS (int[][] board,
2  int startR, int startC, int destR, int destC){
3  Stack<int[]> dfsStack = new Stack<>();
4  boolean[] visited =
5  new boolean[board.length][board[0].length];
6
7
8  dfsStack.push(new int[]{startR, startC});
9
10 while (!dfsStack.isEmpty()) {
11     int[] current = dfsStack.pop();
12     int currentRow = current[0];
13     int currentColumn = current[1];
14
15     if (currentRow == destR
16         && currentColumn == destC) {
17         return new PathResult(true, 0);
18     }
19
20     visited[currentRow][currentColumn] = true;
21
22     // Valid connected neighbours are added here
23 }
24
25 return
26 new PathResult(false, smallestDistance);
27 }

```

Listing 2: Depth-first-search used to check whether a train has reached its destination.

The program uses an iterative depth-first search with a stack. It starts from the train position, follows valid connected tiles, and stops when the assigned destination is reached.

3 GENETIC ALGORITHM

A genetic algorithm was used to search for a tile setting that connects every train to its assigned destination. Each candidate solution represents one complete board. The algorithm begins with a population of candidate boards, evaluates their fitness, keeps the better solutions, and creates new candidates through mutation. This process is repeated for a fixed number of generations, while the highest-scoring board found so far is kept.

3.1 Population and Initialization

The initial population is created from the board that is currently displayed to the user. The first candidate is the unchanged copy of this initial board, whereas the remaining candidates are generated by applying random mutation to the initial board itself. By this, the genetic algorithm can continue from any manual user changes instead of starting from the original board.

3.2 Fitness Function

Each candidate board is assigned a fitness score that represents the quality of the solution. The score is increased when neighboring tiles form valid connections and when a train reaches its destination. The score is decreased when the tiles are changed, when more complex tile types are used, and when a train cannot reach its destination. Boards with higher scores are considered better solutions. The quality of the path is also considered when a train is incapable of reaching its destination. During the depth-first-search, the smallest distance reached between the explored railway path and the destination is recorded. Candidates that bring a train closer to the target destination receive a smaller penalty than those that are further away. This gives the genetic algorithm better guidance before a complete and correct route is found.

3.3 Selection and Mutation

After all the candidates are evaluated, the population is sorted according to fitness. The better candidates are selected as parents for the next generation. New candidates are generated by copying selected boards and randomly changing some tiles. The best board found so far is copied directly to the next generation, so good solutions are recorded and not lost. To prevent the searching from remaining at the same unchanged spot for too long, the algorithm keeps track of how many generations have passed without an improvement in the best fitness score. When this limit has been reached, part of the population is regenerated using new mutations. This provides the algorithm with the capability of searching beyond a previously stable result.

4 SEQUENTIAL IMPLEMENTATION

In the sequential implementation, the candidate boards are evaluated sequentially on a single thread. For each generation, the algorithm goes through the entire population and calls the fitness function for each candidate board. The next candidate waits for evaluation until the evaluation for its previous board is done. After all candidate boards are evaluated with the fitness function, the algorithm identifies the best candidate, selects the parent boards, and creates the next population through the previously mentioned mutation. This process repeats for a specified amount of generations. The supreme board found during this search is stored and displayed when the algorithm is done. The sequential implementation is the baseline for the other implementations. It does not use thread management nor message passing, so it has little execution overhead, meaning it will perform better on smaller boards and populations. Its execution time is later compared with the parallel and distributed versions under the same input parameters and random seed.

```

1  for(int i=0;i<populationSize;i++){
2      fitness[i]=evaluator.evaluateTiles
3      (initialBoard, population[i],
4      trains, trainPairCount);
5      if(fitness[i]>bestScore){
6          bestScore=fitness[i];
7          bestBoard=copyBoard(population[i]);
8      }
9  }
10 }
```

Listing 3: Sequential evaluation of the population.

Listing 3 shows that candidate boards are evaluated one after another. The best score and corresponding board are updated whenever an improved candidate is found.

5 PARALLEL IMPLEMENTATION

The parallel implementation does the fitness evaluation of the candidate boards concurrently. Each candidate can be scored independently, so the population is divided across multiple tasks and processed using a java Executor Service with a fixed thread pool. The number of thread workers is chosen according to the number of available processors, while also ensuring that it does not exceed population size. For each generation, one task is submitted for every candidate board. Each task evaluates the fitness of one candidate and stores the result in the corresponding position of the fitness array. After submitting all tasks, the main thread waits for every Future to complete before continuing with the genetic algorithm. Once all fitness values have been collected, the program updates the best solution, does tournament selection, combines parent boards, using row-based crossover and applies mutations. These steps are the same as in the sequential implementation, only the fitness evaluation phase is parallelized. The best board is copied to the next generation. If fitness does not improve for more than thirty generations, half of the population is regenerated from the current starting board for diversity. Parallel execution has overhead from creating tasks, scheduling tasks, waiting for task completion and result collection. So, it may be slower for small workloads, while larger populations and boards benefit from parallel fitness evaluation. Listing 4 shows how one evaluation task is submitted for each candidate. The main thread waits for all Future objects before continuing with the next genetic algorithm operations.

```

1  int threadCount = Math.max(1,
2  Math.min(populationSize,
3  Runtime.getRuntime().availableProcessors()));
4
5  ExecutorService pool =
6  Executors.newFixedThreadPool(threadCount);
7
8  List<Future<?>> futures =
9  new ArrayList<>(populationSize);
10
11
12
13  for (int i = 0; i < populationSize; i++) {
14
```

```

15     final int idx = i;
16     futures.add(pool.submit(() -> {
17
18         fitness[idx] =
19             evaluator.evaluateTiles(
20                 initialBoard,
21                 currentPopulation[idx],
22                 trains,
23                 trainPairCount
24             );
25     }));
26 }
27
28 for (Future<?> future : futures) {
29     future.get();
30 }

```

Listing 4: Parallel fitness evaluation using a fixed thread pool.

6 DISTRIBUTED IMPLEMENTATION

In distributed implementation, the program uses MPJ Express to divide fitness among multiple MPI processes. Rank 0 acts as the controller, while the remaining ranks act as workers. Rank 0 also starts the graphical interface and manages the genetic algorithm. At the start of the execution, rank 0 sends the board dimensions, the original board, and the train data for every worker. Since MPI messages use one-dimensional arrays, the two-dimensional board and the train data is flattened before transmission. Each worker then reconstructs them locally. For every generation, rank 0 divides the population into batches and sends one batch to each worker that is available. Every worker reconstructs the candidate boards in its batch, evaluates them through the fitness function, and returns the calculated scores together with the starting index of the batch. Rank 0 places the returned scores into the fitness array and then continues with selection, crossover, mutation, and preserving the best solution. So, only population evaluation is distributed, while the remaining genetic algorithm operations are done by rank 0.

```

1  MPI.COMM_WORLD.Send(batchInfo, 0, 2,
2  MPI.INT, worker, 6);
3
4  MPI.COMM_WORLD.Send(flatBatch, 0,
5  flatBatch.length, MPI.INT, worker, 3);
6
7  int[] batchResults = new int[count + 1];
8
9  MPI.COMM_WORLD.Recv(batchResults, 0,
10 batchResults.length, MPI.INT, worker, 4);
11
12 int returnedStartIndex = batchResults[0];
13
14 for (int b=0; b<count; b++){
15     fitness[returnedStartIndex+b] =
16     batchResults[b+1];
17 }
18

```

Listing 5: Rank 0 distributing candidate batches and collecting the fitness values.

Listing 5 shows how rank 0 sends a flattened batch to a worker and receives the corresponding fitness values. The returned starting index is used to place each result into the correct position of the fitness array.

```

1  MPI.COMM_WORLD.Recv(
2  batchInfo,
3  0,
4  2,
5  MPI.INT,
6  0,
7  6);
8
9  int startIndex=batchInfo[0];
10 int count = batchInfo[1];
11 int[] flatBatch =
12 new int[count*cells];
13
14 MPI.COMM_WORLD.Recv(
15 flatBatch,
16 0,
17 flatBatch.length,
18 MPI.INT,
19 0,
20 3
21 );
22
23 int[] batchResults =
24 new int[count+1];
25
26 batchResults[0] = startIndex;
27 for (int b=0; b<count; b++){
28     int[][] candidateBoard =
29     new int[rows][cols];
30
31     for (int r=0; r<rows; r++){
32         for (int c=0; c<cols; c++){
33             int flatIndex =
34             b*cells + r*cols+c;
35
36             candidateBoard[r][c] =
37             flatBatch[flatIndex];
38         }
39     }
40
41     int candidateFitness =
42     evaluator.evaluateTiles
43     (initialBoard, candidateBoard,
44     trains, trainPairCount);
45
46     batchResults[b+1] = candidateFitness;
47 }
48 MPI.COMM_WORLD.Send(
49 batchResults,
50 0,
51 batchResults.length,
52 MPI.INT,
53 0,
54 4 );

```

Listing 6: Worker evaluation of a received candidate batch.

Listing 6 shows how a worker receives a flattened batch, reconstructs the boards, evaluates each fitness, and returns the results to rank 0.

7 EXPERIMENTAL SETUP

The experiments were done on a MacBook Air equipped with an Apple M2 processor, an 8-core CPU consisting of four performance cores and four efficiency cores, and 8GB of unified memory. The computer was running on macOS Tahoe 26.5.2. The project was implemented in Java, while MPJ Express 0.44 was used for distributed implementation. To guarantee a fair comparison, the same fixed random seed and the same 10×10 board with 6 train-destination pairs were used for all three versions. Three workloads consisted the testing: a small workload with a population of 100 and 200 generations, a medium workload with 250 population and 500 generations, and finally a large workload with a population of 500 and 1000 generations. Execution time is reported in milliseconds.

Table 1: Workloads in experiments

Workload	Grid size	Train pairs	Population size	Generations
Small	10×10	6	100	200
Medium	10×10	6	250	500
Large	10×10	6	500	1000

For the parallel test, the large workload was done with 1, 2, 4 and 8 threads. For the distributed test, the large workload was done using 2, 4, 8 and 16 MPI ranks. Rank 0 was the controller, the remaining ranks evaluated candidate boards.

7.1 Additional Controlled Experiments

In addition to the workload comparison, two more tests were performed. First, the effect of board size was tested by keeping the train pairs fixed at 2 and using board sizes of 6×6 , 8×8 , and 10×10 . Additionally, the effect of the number of train pairs was tested by keeping the board sizes fixed to 10×10 , and using 2, 4 and 6 train-destination pairs. In both tests, the population size was 100 and generations were at 200. Each case was run in sequential, parallel with 8 threads, and distributed version with 4 MPI ranks.

8 RESULTS AND DISCUSSION

All three implementations returned the same final fitness value and the same fitness progression for each of the workloads. This is a confirmation that the parallelization and distribution of the fitness evaluation did not change the behavior of the genetic algorithm. The execution times differed, all depending on workload and number of threads or ranks.

Table 2 shows that the relative performance of the implementations depends on the size of the workload. For the small workload, the sequential implementation was faster than the parallel implementation with 8 threads because the workload was not enough to compensate the thread-management overhead. However, the distributed implementation with 4 ranks had the lowest execution time. With the increasing of the workload, the parallel implementation reduced the execution time from 5495.27 ms to 2322.05 ms, while the distributed version of four ranks obtained 2377.25 ms.

Table 2: Execution time and final fitness for the three workloads

Workload	Sequential (ms)	Parallel (8) (ms)	Distributed(4) (ms)	Fitness
Small	372.08	416.98	278.20	1243
Medium	1391.51	844.0	917.38	1275
Large	5496.27	2322.05	2377.25	1287

Table 3: Caption

Figure 2 shows that by increasing the number of threads we get

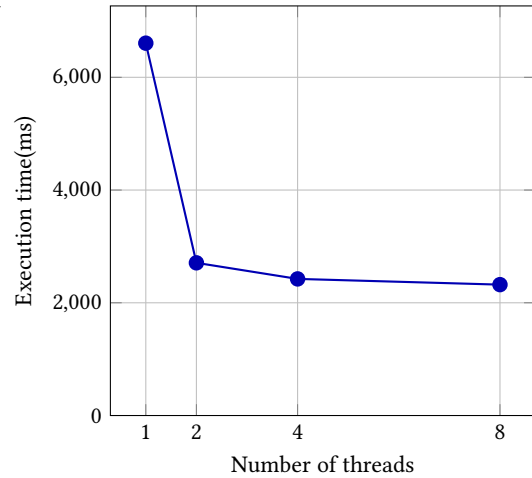


Figure 2: Parallel execution time for the large workload using different thread counts.

reduced execution time of large workload. The largest improvements occurred when the thread count was increased from 1 to 2, and from 2 to 4. Increasing to 8 threads gave small gain, which indicates diminishing returns because of thread-management and overhead scheduling. The parallel version with one thread was slower than the sequential implementation because it had overhead from the parallel setup without having any actual benefits from the concurrent execution.

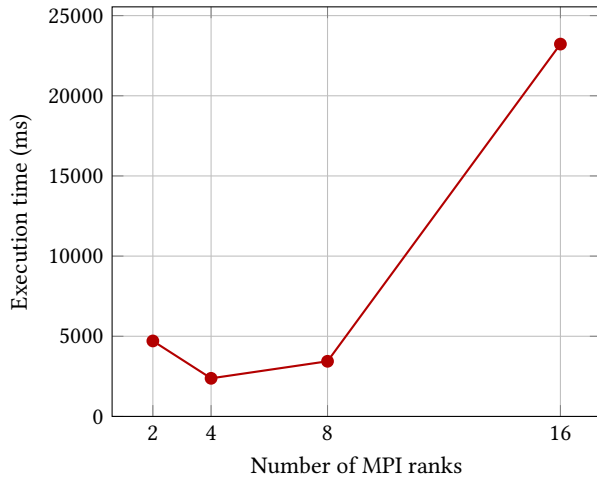


Figure 3: Distributed execution time for the large workload using different MPI ranks

Figure 3 shows that the best distributed performance was done with 4 MPI ranks. Increasing the number of ranks from 2 to 4 reduced execution time, but using 8 ranks made it slower. A 16 rank configuration gave much bigger runtime because of process management and communication overhead which outweighed the benefit of the additional workers. This shows that adding more MPI ranks does not always imply improved performance, especially when all of the ranks are run on one machine.

Table 4: Execution time for different board sizes with 2 trains

Board size	S(ms)	P, 8 threads (ms)	D, 4 ranks (ms)
6 × 6	139.65	286.60	165.92
8 × 8	198.07	329.06	271.30
10 × 10	279.63	363.28	285.94

S = Sequential, P = Parallel, D = Distributed.

Table 4 shows that the execution time increased as the board size increased. The sequential version was the fastest for all three board sizes because the workload is relatively small and the overhead of threads and MPI processes was not compensated with the amount of computation. The parallel implementation had the highest overhead, whereas the distributed version with 4 ranks remained close with performance to the sequential version for the largest board.

Table 5: Execution time for different number of trains on a 10 x 10 board

Train pairs	S (ms)	P, 8 threads (ms)	D, 4 ranks (ms)
2	279.63	363.28	285.94
4	325.37	386.32	284.24
6	372.08	416.98	278.20

S = Sequential, P = Parallel, D = Distributed.

Table 5 shows that sequential and parallel execution times generally increased as the number of trains increased because each candidate board required more path evaluations. The distributed version remained relatively stable and even decreased slightly. The small differences between the distributed runtimes are not large enough to show a clear trend. Overall, increasing the number of train pairs added more evaluation work, which was most clear in the sequential and parallel implementations.

Overall the experiments show that the most suitable mode depends on the workload and the board size. For a small workload, parallel was slower than sequential, while 4 ranks in distributed achieved the best time. For medium and large workloads, parallel execution became more effective, since 8 threads produced the best performance. The distributed version improved performance using 4 ranks, but additional ranks created more overhead instead of speed. The additional tests showed that execution time generally increased by increasing the board size and number of train destination pairs. The increase was most clear in the sequential and parallel versions, while the distributed runtimes remained relatively stable. These results show that the program benefits from concurrency only when the amount of work is large enough to overpower the cost of scheduling, coordination and communication.

9 CONCLUSION

The project successfully implemented the Railroads problem in sequential, parallel and distributed versions. All three versions produced the same final fitness values for the same board and random seed, confirming that the parallel and distributed versions did not corrupt the behavior of the genetic algorithm. The results depicted that parallel and distributed versions were more useful for medium and large workloads, while smaller workloads were affected by thread and processing overhead. For the large workload the best parallel result was done using 8 threads, while the best distributed result was done by 4 MPI ranks. Increasing the number of threads beyond this point reduced performance. The additional experiments also showed that execution time generally increased when the board size or the number of trains increased. This confirms that larger problems require more work during fitness evaluation, which creates more opportunities for parallel and distributed versions to be beneficial. Future work could focus on reducing communication and process management overhead in the distributed version and handling how the candidate boards are assigned to worker ranks in a more suitable manner that would lead to performance improvement. The genetic algorithm could be strengthened by experimenting with different mutation, crossover and selection strategies. The results show that selecting an appropriate number of thread or ranks is as important as using parallel or distributed execution itself, since an excessiveness of concurrency could also reduce performance, instead of improving it.

REFERENCES

- [1] University of Primorska, FAMNIT. Railroads: An Evolutionaty Approach to Problem Solving. Available at: <https://prog3.student.famnit.upr.si/projects/GArailroads/>.
- [2] Gregory R. Andrews. *Foundations of Multithreaded, Parallel, and Distributed Programming*. Addison-Wesley, 100/
- [3] MPJ Express Project. *MPJ Express: An Implementation of MPI in Java* Available at: <http://mpjexpress.org/docs/guides/linuxguide.pdf>
- [4] Oracle. ExecutorService Interface, Java Available at: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/ExecutorService.html>